

Java Backend Architecture

Interview Series

Chapter 13.7

Load Testing

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Load Testing

Load testing is the practice of deliberately driving a system with simulated user traffic to measure how it behaves under expected and extreme conditions. A well-designed load test validates that a service meets its Service Level Agreements (SLAs) for throughput (requests per second), latency percentiles (p50/p95/p99), and error rate before production traffic arrives. Different test types serve different purposes: load tests verify normal-traffic behaviour, stress tests find the breaking point, spike tests validate burst handling, soak tests expose memory leaks and slow degradation, and volume tests check large-data-set performance.

The Java ecosystem offers several tools: Gatling (Scala/Java DSL, first-class open model support), k6 (JavaScript DSL, modern CLI tool), JMeter (GUI-based, widely supported), and wrk/wrk2 (CLI HTTP benchmarking). A critical concept is the open vs closed load model: open models emit requests at a fixed arrival rate (matching real internet traffic) and expose true queuing behaviour; closed models only send a new request when the previous completes, hiding latency under load. The coordinated omission problem — identified by Gil Tene — explains why closed models under-report high-percentile latency. Understanding these distinctions is essential for designing honest load tests and interpreting results correctly.

Load Test Type Profiles

Load Test Type Profiles



Little's Law: $N = \lambda * W$

N = avg concurrent users in system

λ = avg arrival rate (RPS) W = avg response time (seconds)

Example: 100 concurrent users, 200ms response $\Rightarrow \lambda = 100/0.2 = 500$ RPS

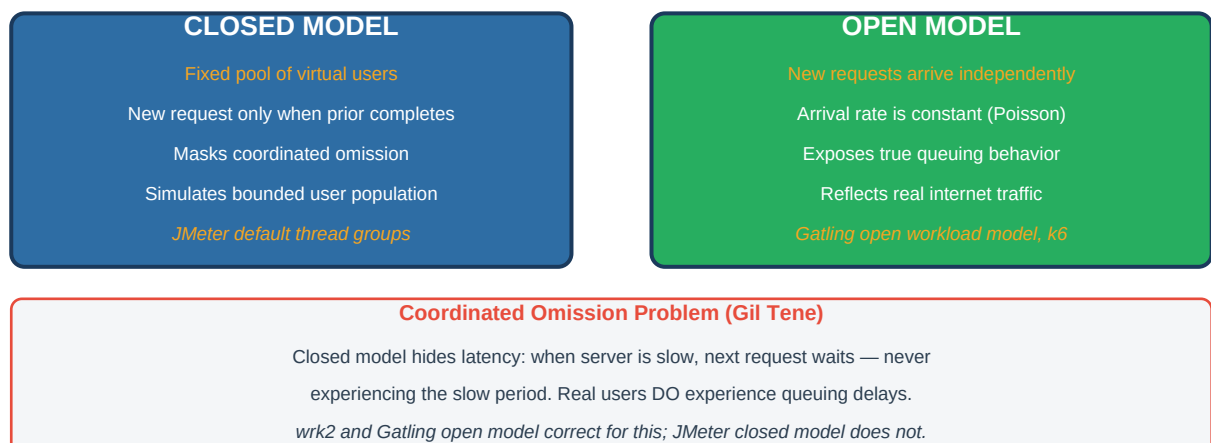
Key Definitions

Load Test	Simulates expected peak traffic to verify the system meets SLA targets for throughput and latency.
Stress Test	Increases load beyond expected capacity to identify the breaking point and observe failure behaviour.
Spike Test	Applies a sudden massive burst of traffic to test auto-scaling and burst handling, then returns to normal.
Soak Test	Runs at sustained load for hours or days to detect memory leaks, connection pool exhaustion, or gradual degradation.

Volume Test	Tests with large datasets to ensure performance does not degrade with data growth (index scans, pagination).
Throughput (RPS)	Requests per second successfully processed. The primary measure of system capacity.
Latency Percentile	p99 = 99% of requests complete within this time. p50 is median. p99 and p999 reveal tail latency issues.
Little's Law	$N = \lambda * W$. Avg concurrency = arrival rate * avg response time. Fundamental queuing theory relationship.
Open Load Model	New requests arrive at a fixed rate regardless of in-flight requests. Exposes true queuing delay. Gatling, k6.
Closed Load Model	A fixed pool of virtual users; next request starts only when previous completes. Hides coordinated omission.
Coordinated Omission	Measurement artifact in closed models where slow responses prevent new requests during the slow period, hiding true latency tail.
Ramp-up	Gradually increasing load at test start to avoid a thundering herd. Allows JIT warm-up and connection pool fill.
Think Time	Simulated pause between user actions modelling realistic user behaviour. Critical for accurate concurrency modelling.
SLA	Service Level Agreement defining performance targets: e.g., p99 < 200ms and error rate < 0.1% at 1000 RPS.
Saturation Point	The load level beyond which response time grows nonlinearly and error rate climbs. The system capacity limit.

Open vs Closed Load Model

Open vs Closed Load Model

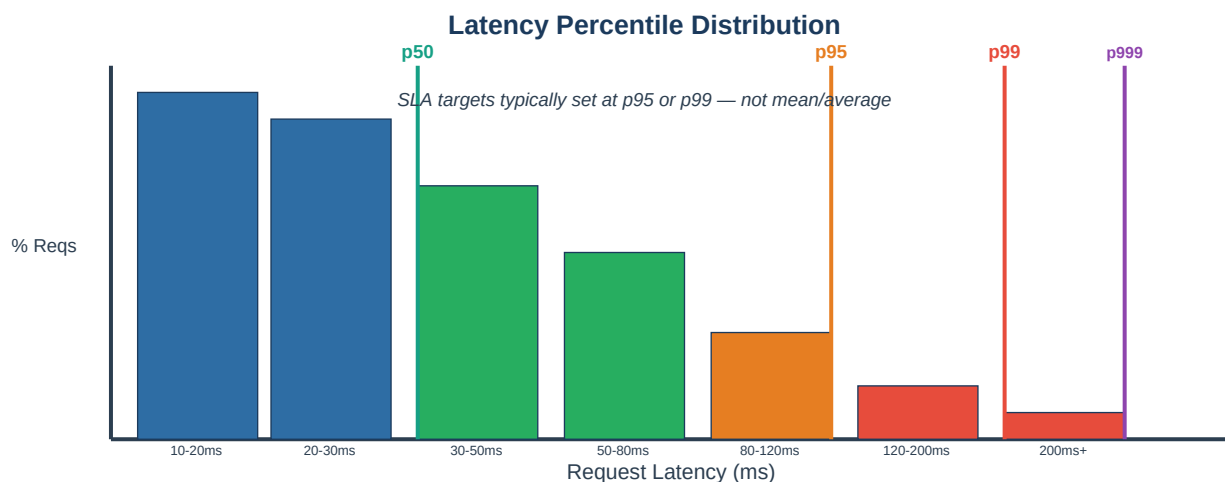


Tool Comparison

Tool	Language	Model	Protocol	CI/CD	Best For
------	----------	-------	----------	-------	----------

Gatling	Scala/Java DSL	Open+Closed	HTTP/WS/gRPC	Maven/Gradle plugin	Complex scenarios, reports
k6	JavaScript	Open	HTTP/WS/gRPC	CLI + cloud	Modern microservices, CI
JMeter	GUI + XML	Closed	HTTP/JDBC/JMS	CLI headless mode	Enterprise, legacy suites
wrk	CLI / Lua	Open	HTTP only	Shell script	Quick HTTP benchmarking
wrk2	CLI / Lua	Open	HTTP only	Shell script	Corrected latency (HDR)
Locust	Python	Open	HTTP/custom	CLI	Pythonic load testing

Latency Percentile Distribution



Annotated Code Examples

1. Gatling simulation — open model with ramp-up

```
import io.gatling.javaapi.core.*; import io.gatling.javaapi.http.*; import static
io.gatling.javaapi.core.CoreDsl.*; import static io.gatling.javaapi.http.HttpDsl.*; public
class OrderSimulation extends Simulation { HttpProtocolBuilder httpProtocol = http
.baseUrl("https://api.example.com") .acceptHeader("application/json")
.contentTypeHeader("application/json"); ScenarioBuilder scn = scenario("Create Order")
.exec(http("POST /orders") .post("/orders") .body(StringBody("{\"productId\":\"123\",\"qty\":1}"))
.check(status().is(201)) .check(jsonPath("$.orderId").saveAs("orderId"))) .pause(1) // think
time .exec(http("GET /orders/{orderId}") .get("/orders/#{orderId}") .check(status().is(200)));
// Open model: inject 100 RPS over 5 minutes, ramp from 0 { setUp(scn.injectOpen(
rampUsersPerSec(0).to(100).during(60), // ramp up constantUsersPerSec(100).during(240), //
sustain rampUsersPerSec(100).to(0).during(30) // ramp down ).protocols(httpProtocol))
.assertions( global().responseTime().percentile(99).lt(500), // p99 < 500ms
global().successfulRequests().percent().gt(99.0) // > 99% success ); } }
```

2. k6 script — open model with thresholds

```
// k6 run --vus 50 --duration 5m script.js import http from "k6/http"; import { check, sleep }
from "k6"; import { Rate, Trend } from "k6/metrics"; const errorRate = new Rate("errors");
const orderDuration = new Trend("order_duration", true); export const options = { scenarios: {
open_model: { executor: "constant-arrival-rate", // open model rate: 100, // 100 RPS timeUnit:
"1s", duration: "5m", preAllocatedVUs: 50, maxVUs: 200, }, }, thresholds: { http_req_duration:
["p(99)<500"], // p99 < 500ms "http_req_duration{name:orders}": ["p(95)<300"], errors:
```

```
[ "rate<0.01"], // error rate < 1% }, }; export default function () { const res =
http.post("https://api.example.com/orders", JSON.stringify({ productId: "123", qty: 1 })), {
headers: { "Content-Type": "application/json" }, tags: { name: "orders" } });
errorRate.add(res.status !== 201); orderDuration.add(res.timings.duration); check(res, {
"status 201": (r) => r.status === 201 }); sleep(1); }
```

3. JMeter — headless CLI execution

```
# Run JMeter test plan headlessly and generate HTML report jmeter -n \ -t test_plan.jmx \ -l
results.jtl \ -e -o /tmp/html_report \ -Jthreads=100 \ -Jrampup=60 \ -Jduration=300 # Key
JMeter properties via -J flags # threads = number of virtual users (closed model) # rampup =
seconds to ramp all threads up # duration = test duration in seconds # Maven plugin for CI
integration (pom.xml) # <plugin> # <groupId>com.lazerycode.jmeter</groupId> #
<artifactId>jmeter-maven-plugin</artifactId> # <configuration> #
<testResultsTimestamp>>false</testResultsTimestamp> # <propertiesUser> # <threads>100</threads>
# <duration>300</duration> # </propertiesUser> # </configuration> # </plugin>
```

4. wrk2 — corrected latency benchmarking

```
# wrk2 maintains a CONSTANT throughput rate (open model) # and uses HDR histogram for accurate
high-percentile latency # 100 connections, 10 threads, 60s, 10000 RPS target rate wrk2 -c 100
-t 10 -d 60s -R 10000 \ --latency \ http://localhost:8080/api/orders # Sample output: # Running
1m test @ http://localhost:8080/api/orders # 10 threads and 100 connections # Latency
Distribution (HdrHistogram) # 50.000% 8.23ms # 75.000% 12.11ms # 90.000% 18.45ms # 99.000%
45.23ms <- SLA target # 99.900% 123.45ms <- tail latency # 99.990% 456.78ms <- outliers #
Requests/sec: 9987.34 # Transfer/sec: 3.21MB # wrk (without rate control) – simple throughput
test wrk -c 100 -t 10 -d 30s http://localhost:8080/api/health
```

5. Gatling Maven plugin CI/CD integration

```
<!-- pom.xml: Gatling Maven plugin --> <plugin> <groupId>io.gatling</groupId>
<artifactId>gatling-maven-plugin</artifactId> <version>4.9.0</version> <configuration>
<simulationClass>com.example.OrderSimulation</simulationClass>
<resultsFolder>${project.build.directory}/gatling</resultsFolder> </configuration>
<executions> <execution> <id>performance-test</id> <phase>verify</phase> <!-- runs during
Maven verify phase --> <goals><goal>test</goal></goals> </execution> </executions> </plugin>
// Assertions fail the Maven build when SLA is breached
setUp(scn.injectOpen(constantUsersPerSec(50).during(120)) .protocols(httpProtocol))
.assertions( global().responseTime().percentile(95).lt(300),
global().responseTime().percentile(99).lt(1000),
global().successfulRequests().percent().gt(99.5), global().requestsPerSec().gt(45.0) // at
least 45 RPS achieved );
```

6. Spring Boot actuator metrics during load test

```
// Monitor live JVM metrics via Actuator during load test // GET
/actuator/metrics/http.server.requests // GET /actuator/metrics/jvm.gc.pause // GET
/actuator/metrics/hikaricp.connections.pending // Prometheus query to check p99 during load
test // histogram_quantile(0.99, // rate(http_server_requests_seconds_bucket[1m])) // Custom
health indicator for load test readiness @Component public class LoadTestHealthIndicator
implements HealthIndicator { private final HikariDataSource ds; @Override public Health
health() { int pending = ds.getHikariPoolMXBean().getThreadsAwaitingConnection(); int active =
ds.getHikariPoolMXBean().getActiveConnections(); if (pending > 10) { return Health.down()
.withDetail("pending_connections", pending) .withDetail("active_connections", active)
.build(); } return Health.up() .withDetail("active_connections", active) .build(); } }
```

Interview Q&A;

Q1. What are the five main types of load tests and when do you use each?

Load test: verifies the system meets SLAs at expected peak traffic. Stress test: pushes beyond capacity to find the breaking point and observe failure modes. Spike test: applies a sudden burst to test auto-scaling and burst handling. Soak test: runs at sustained load for hours or days to detect memory leaks, connection pool exhaustion, or slow degradation. Volume test: tests with large datasets to ensure performance does not degrade with data growth.

Q2. What is Little's Law and how is it applied in load testing?

Little's Law states $N = \lambda * W$, where N is the average number of concurrent requests in the system, λ is the arrival rate (RPS), and W is the average response time. It allows you to calculate required concurrency: at 500 RPS with 200ms average latency, you need 100 concurrent workers. It also lets you verify your load generator: if you inject 100 VUs with 1s think time at 200ms response, expected RPS is $100/(0.2+1) = \sim 83$ RPS.

Q3. What is the difference between the open and closed load model?

In a closed model a fixed pool of virtual users operates; a new request starts only when the previous completes. In an open model requests arrive at a fixed rate (e.g., Poisson process) independent of in-flight requests. Open models reflect real internet traffic. Closed models can hide latency: when the server slows, virtual users wait, reducing arrival rate and masking queuing delays. Gatling and k6 support open models; JMeter defaults to closed.

Q4. What is the coordinated omission problem?

Coordinated omission (identified by Gil Tene) is a measurement artifact in closed load models. When the server experiences a slow period, the closed-model load generator stops sending new requests (all VUs are waiting for slow responses). The load test never experiences the queuing delay that real users would face during the slow period, so high-percentile latencies are drastically underreported. Open-model tools (wrk2, Gatling open injection) correct for this.

Q5. What metrics should you capture during a load test?

Throughput (RPS achieved vs target), latency percentiles (p50, p95, p99, p999), error rate (% of failed responses), saturation indicators (CPU %, thread pool utilisation, connection pool pending, GC pause time), and resource exhaustion signals (heap memory, open file descriptors, network buffers). SLA targets are typically set at p99 or p95, never at mean/average, because mean hides tail latency.

Q6. How does Gatling differ from JMeter?

Gatling uses a reactive, non-blocking architecture (Akka + Netty) supporting thousands of concurrent users on a single machine. Tests are written as Scala/Java code (not XML), enabling version control and code review. Gatling has first-class open model support (constantUsersPerSec, rampUsersPerSec). JMeter uses a thread-per-user model (limits scale), GUI-based XML test plans, and defaults to a closed model. Gatling generates richer HTML reports; JMeter has a larger plugin ecosystem.

Q7. What is a ramp-up period and why is it important?

A ramp-up period gradually increases load from zero to the target rate at the start of a test. It is important for: (1) avoiding a thundering herd that would overwhelm the server instantly; (2) allowing JVM JIT compilation to warm up (avoiding cold-start latency bias); (3) allowing connection pools to fill; (4) allowing DNS and CDN caches to warm. Without ramp-up, the first minutes of results reflect cold-start behaviour, not steady-state performance.

Q8. How do you handle test data management in load tests?

Feed unique test data per virtual user to avoid cache hits masking real latency. Gatling uses csv feeders; JMeter uses CSV Data Set Config. Strategies: (1) pre-generate a large dataset (e.g., 100k unique user IDs, product IDs); (2) use random or circular feeders to distribute load; (3) avoid database conflicts by partitioning data ranges per thread; (4) clean up test data after runs in non-production environments. Production load tests should use shadow traffic or traffic mirroring.

Q9. What is think time and how does it affect load model accuracy?

Think time is the simulated pause between user actions, representing the time a real user spends reading a page before clicking. Including realistic think time (0.5-3s) makes VU concurrency more accurate per Little's Law. Without think time, a fixed number of VUs generates much higher RPS (users never pause), resulting in an unrealistically closed model. Think time should be derived from real user session recordings (e.g., from browser performance tools or access logs).

Q10. How do you integrate load tests into CI/CD?

Run load tests as part of the performance test stage in the pipeline (separate from unit/integration tests due to duration). Gatling Maven plugin runs simulations during the verify phase and fails the build when assertions are breached. k6 returns non-zero exit code on threshold failures. Store results (RPS, p99) as build artifacts and compare against a baseline. Trigger performance tests nightly or on release branches, not every commit. Alert when regressions exceed 10-20% of baseline.

Q11. What is saturation in the context of load testing?

Saturation is the point at which a resource (CPU, thread pool, connection pool, memory) is fully utilised and additional load causes queuing rather than proportional throughput increase. Beyond the saturation point, latency grows nonlinearly and error rate climbs. In USL (Universal Scalability Law) analysis, saturation shows as the throughput curve bending then declining. Identifying the saturation point helps size infrastructure and set capacity planning headroom (typically 60-70% utilisation at peak).

Q12. What is the difference between throughput and concurrency?

Throughput (RPS) is the number of requests completed per second — the output rate of the system. Concurrency is the number of requests simultaneously in flight — this determines the required parallelism in thread pools and connection pools. By Little's Law: $\text{concurrency} = \text{RPS} * \text{average latency}$. At 1000 RPS and 50ms latency, you need 50 concurrent threads minimum. High concurrency with low RPS means high latency; high RPS with low concurrency means fast responses.

Q13. How do you measure and interpret latency percentiles?

p50 (median) is the typical user experience. p95 is the SLA boundary for 95% of users. p99 is the tail — 1 in 100 requests experiences this latency or worse. p999 (p99.9) reveals extreme outliers, important for systems handling millions of RPS (1000 users per second experience p999 latency). Always use percentiles not mean/average for SLA targets, because averages are skewed by outliers and hide tail latency. HDR histograms (wrk2, HdrHistogram library) provide accurate percentile computation.

Q14. What is wrk2 and how is it different from wrk?

wrk2 is a fork of wrk that adds a constant throughput rate option (-R flag). Standard wrk sends requests as fast as possible (closed model — no rate control), making it unsuitable for measuring latency under a specific load level. wrk2 maintains a precise target RPS using coordinated-omission-corrected HDR histograms, delivering accurate p99/p999 latency at a specified throughput level. It is ideal for "what is my latency at exactly 10,000 RPS?" questions.

Q15. What does an SLA look like in a load test context?

A well-defined load SLA specifies: (1) load level: at 500 RPS sustained for 10 minutes; (2) latency: p95 < 200ms, p99 < 500ms; (3) error rate: < 0.1%; (4) saturation: CPU < 80%, connection pool pending < 5. Gatling

assertions express this as: `global().responseTime().percentile(99).lt(500), global().successfulRequests().percent().gt(99.9)`. Failed assertions cause the build to fail in CI/CD.

Q16. How do you simulate realistic user behaviour in a load test?

Capture real session flows from production access logs or browser recording tools. Implement scenarios that mirror real user journeys (login, browse, add to cart, checkout). Use parameterised requests with real user IDs, product IDs from data feeders. Add realistic think times between steps. Include a mix of read-heavy and write requests reflecting production traffic distribution. Use Gatling feeders or k6 shared arrays for test data; avoid hardcoded values that result in 100% cache hits.

Q17. What is test data isolation in load testing?

Test data isolation ensures load test traffic does not contaminate production data and vice versa. Strategies: use a dedicated performance environment with production-like data volumes; for staging, use synthetic data seeded before the test; tag load test requests with headers (X-Load-Test: true) for filtering from analytics. For database tests, use separate schemas or prefix synthetic entity IDs to enable easy cleanup post-test.

Q18. How do you identify a connection pool bottleneck during a load test?

Monitor HikariCP metrics via `/actuator/metrics/hikaricp.connections.pending` (threads waiting for a connection). If this climbs above 0 sustained, the pool is undersized for the current RPS and latency combination. Also check `hikaricp.connections.acquire` (acquisition time). In Grafana, a rising pending connections counter while p99 latency spikes indicates connection pool saturation. Fix: increase pool size (within DB max_connections budget) or reduce query latency.

Q19. What is the Universal Scalability Law (USL)?

USL models system throughput as a function of concurrency N , incorporating two penalties: contention (σ) — sequential resource sharing — and coherency (κ) — coordination overhead (e.g., distributed locks, cache invalidation). $\text{Throughput} = N / (1 + \sigma * (N-1) + \kappa * N * (N-1))$. USL analysis of load test data reveals whether a system scales linearly, has contention issues, or will retrograde (throughput decreases as concurrency grows). High κ indicates distributed coordination bottlenecks.

Q20. What is a baseline load test and why is it important?

A baseline load test establishes performance reference metrics (RPS, p50/p99 latency, error rate, CPU/memory) for a known version of the application. All future load test results are compared against this baseline to detect regressions. Without a baseline, you cannot tell whether a p99 of 350ms is good or bad. Baselines should be stored as artifacts (JSON/CSV) and automatically compared in CI/CD pipelines.

Q21. How do you test auto-scaling behaviour with a spike test?

Design a spike test that rapidly increases RPS by 5-10x over 30-60 seconds, sustains the spike for 2-3 minutes, then drops back. Measure: (1) time to scale out (first new instance healthy and receiving traffic); (2) error rate during scale-out (requests failing before new instances are ready); (3) time to scale in after load drops; (4) latency behaviour during scale events. Compare against auto-scaling policy thresholds (e.g., scale out when CPU > 70% for 2 minutes). Adjust scale-out trigger thresholds or increase min instances if spike causes excessive errors.